SPICED+: Syntactical Bug Pattern Identification and Correction of Trojans in A/MS Circuits Using LLM-Enhanced Detection

Jayeeta Chaudhuri, *Member, IEEE*, Dhruv Thapar, Arjun Chaudhuri, Farshad Firouzi, and Krishnendu Chakrabarty, *Fellow, IEEE*

***Abstract*— Analog and mixed-signal (A/MS) integrated circuits (ICs) are crucial in modern electronics, playing key roles in signal processing, amplification, sensing, and power management. Many IC companies outsource manufacturing to third-party foundries, creating security risks such as syntactical bugs and stealthy analog Trojans. Traditional Trojan detection methods, including embedding circuit watermarks and hardware-based monitoring, impose significant area and power overheads while failing to effectively identify and localize the Trojans. To overcome these shortcomings, we present SPICED+, a software-based framework designed for syntactical bug pattern identification and the correction of Trojans in A/MS circuits, leveraging large language model (LLM)-enhanced detection. It uses LLM-aided techniques to detect, localize, and iteratively correct analog Trojans in SPICE netlists, without requiring explicit model training, and thus incurs zero area overhead. The framework leverages chain-of-thought reasoning and few-shot learning to guide the LLMs in understanding and applying anomaly detection rules, enabling accurate identification and correction of Trojan-impacted nodes. With the proposed method, we achieve an average Trojan coverage of 93.3%, average Trojan correction rate of 91.2%, and an average false-positive rate of 1.4%.**

***Index Terms*— Anomaly detection, hardware security, large language models (LLMs), SPICE, Trojan horse.**

I. Introduction

ANALOG and mixed-signal (A/MS) integrated circuits (ICs) play a critical role in signal processing, amplifiers, sensors, and power management systems. Many IC companies outsource the manufacturing and fabrication of their analog designs to third-party foundries to avoid expensive capital expenditures and substantial costs associated with investing in manufacturing infrastructure. The globalization of the semiconductor industry and the outsourcing of analog ICs to third-party vendors have introduced critical security threats, which can be broadly categorized as syntactical bug or analog Trojan [2]. A syntactical bug refers to unintentional syntax errors in an A/MS design (e.g., incorrect component definitions, floating nodes, etc.). In contrast, an analog Trojan is a malicious modification introduced during the design or fabrication phase that notably compromises the integrity of A/MS ICs [3], [4]. A major risk arises from the stealthy embedded Trojans that activate only under specific operating conditions, such as certain input bias voltages, while remaining dormant otherwise [5], [6]. This allows the Trojan to evade detection during traditional testing and verification methods.

Analog Trojans are designed to have negligible area footprint, enabling their easy integration into larger, complex A/MS designs at the netlist level, involving multiple signal paths and transistor components. These stealthy components are activated only at specific operating bias voltages and remain dormant otherwise. For example, the A2 Trojan [5] is activated only during rare trigger conditions such as division-by-zero operation in a microcontroller. Prior work on analog Trojan detection uses current-sensing amplifiers to identify Trojan activation [7]. A recent study introduces a sensitivity analysis-based framework leveraging analog neural twins to detect stealthy analog Trojans [8]. This approach identifies the critical paths most vulnerable to Trojan insertion [9]. Next, circuit watermarks are embedded to monitor deviations in these paths, triggering an alert when a Trojan is activated.

Recent advancements in large language models (LLMs) have showcased their significant capabilities across various software-related tasks such as automated hardware description language (HDL) code generation, code optimization and verification, and test case generation [10], [11]. Advances in the electronic design automation (EDA) field have made LLMs highly relevant for innovative applications in A/MS design, including analog layout generation, netlist generation, and analog circuit sizing [12], [13], [14]. Prior work on Trojan detection, including embedding circuit watermarks or conducting hardware-based monitoring, impose significant area and power overheads, especially in large analog designs [7], [9]. While these methods are effective for detecting Trojans, they do not address the challenges of Trojan localization and mitigation, which become increasingly difficult as circuit complexity grows.

In [1], an LLM-based framework, namely, SPICED, was introduced to detect and localize bugs in SPICE netlists. The framework teaches the LLM a predefined set of supervised learning rules that explain various Trojan behaviors, enabling it to identify anomalies from current and voltage simulation logs.

Received 11 October 2024; revised 21 December 2024; accepted 6 January 2025. Date of publication 15 January 2025; date of current version 24 March 2025. This work was supported by the National Science Foundation under Grant CNS-2310142. An earlier version of this paper was presented at the IEEE Physical Assurance and Inspection of Electronics, 2024 [DOI: 10.1109/PAINE62042.2024.10792717]. *(Corresponding author: Jayeeta Chaudhuri.)*

Jayeeta Chaudhuri, Dhruv Thapar, Farshad Firouzi, and Krishnendu Chakrabarty are with the School of Electrical, Computer, and Energy Engineering, Arizona State University, Tempe, AZ 85287 USA (e-mail: jchaudh3@asu.edu).

Arjun Chaudhuri is with NVIDIA Corporation, Santa Clara, CA 95051 USA.

Digital Object Identifier 10.1109/TVLSI.2025.3527382

1063-8210 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

SPICED operates within the software domain, thereby eliminating the need for any hardware modifications to the analog design. SPICED excels at intelligent parsing and analysis of large volumes of structured data, e.g., from SPICE netlists, enabling accurate identification of Trojan-impacted nodes. However, this approach lacks the ability to accurately identify all Trojan lines within the SPICE netlist, and more critically, to mitigate their impact. This necessitates an enhanced solution that facilitates both the detection and mitigation of Trojans in A/MS designs, while avoiding any area or power overheads.

A preliminary version of this work was presented in [1], where syntactical bug detection and localization were performed (see Section III). In this article, we present a novel LLM-based framework that not only detects and localizes analog Trojans in A/MS designs but also mitigates them, an aspect not addressed in the previous work [1]. This article significantly improves upon [1] by accurately identifying Trojans and iteratively refining the netlist until it is free of Trojans. This capability has been tested on several A/MS circuits, including a bandgap filter and low-dropout voltage regulator (LDO). The key contributions of this article are as follows.

1) *Trojan Mitigation:* We introduce SPICED+, an iterative correction-based flow that transforms [1] to a fully automated detection and mitigation tool.
2) *Incorporating Multiple Tools:* SPICED+ integrates multiple tools (including SPICED and HSPICE) to iteratively remove suspected Trojan lines, rerun HSPICE simulations, and verify whether the design meets the desired specifications.
3) *Confidence Scoring Metric:* We formulate a confidence scoring metric and prompt the LLM to generate confidence scores for each identified Trojan-impacted node. By incorporating confidence scores into the decision-making process, SPICED+ minimizes false positives and ensures that only the actual Trojan components are mitigated.
4) *Evaluation on Larger Circuits:* We evaluate SPICED+ on larger and more complex circuits using the GPT-4o-mini model; our method shows a high true positive rate and Trojan coverage for these circuits. We also report the Trojan diagnosis reports generated by LLM.

The remainder of this article is organized as follows. Section II provides an overview of recent work on LLM-guided code verification and bug detection and how its capability can be leveraged for bug detection in the analog domain. In Section III, we introduce the syntactical bug detection framework. Section IV focuses on LLM-aided Trojan detection and localization. In Section V, we present SPICED+, an automated workflow for Trojan detection and mitigation. Evaluation results for SPICED+ are presented in Section VI. Section VII discusses extensions of the current research on LLMs for analog security. Finally, Section VIII concludes this article.

II. Background and Motivation

A. Design Automation Using LLMs

LLMs have been integrated into various stages of EDA, including tasks such as code generation, placement and routing, as well as security measures such as vulnerability insertion and detection. Recent work used LLM for Verilog code generation by fine-tuning the existing LLM with Verilog datasets [10]. Fine-tuned open-source CodeGen LLM outperformed the state-of-the-art commercial LLM in generating functionally correct designs [18]. Blocklove et al. [16] used LLM-based iterative flow to design an 8-bit accumulator-based microprocessor architecture. Lai et al. [13] proposed LLM for generating analog circuits with a feedback-enhanced flow to enable self-correcting design of analog circuits. The feedback allowed generation of circuits without any LLM training involved. Domain-adaptive pretraining (DAPT) followed by supervised fine-tuning (SFT) of foundation LLM models enabled an assistant chatbot for chip design [17]. Wu et al. [21] and Zhong et al. [11] proposed LLM-based script generation to facilitate the EDA design flow. These efforts are focused mainly in the digital domain, as shown in Table I.

TABLE I
Comparison to Prior LLM-Aided Techniques

Method	Domain	Training-Free	Bug Detection	Bug Fixing	Trojan Detection	Trojan Mitigation
[10]	Digital	✗	✗	✗	✗	✗
[15]	Analog	✗	✗	✗	✗	✗
[13]	Analog	✓	✗	✗	✗	✗
[16]	Digital	✓	✗	✗	✗	✗
[17]	Digital	✗	✗	✗	✗	✗
[18]	Digital	✗	✗	✗	✗	✗
[19]	Digital	✓	✗	✗	✗	✗
[20]	Digital	✓	✓	✓	✗	✗
[1]	Analog	✓	✓	✓	✓	✗
SPICED+	Analog	✓	✓	✓	✓	✓

B. LLMs for Hardware Security

1) Syntactical Bug Correction: In [20], LLM agents are used for iteratively refining erroneous Verilog code until all the syntactical errors are resolved. However, LLM fails to correct syntax errors that require advanced reasoning and problem-solving skills, such as identifying out-of-bounds errors. Ahmad et al. [22] automatically fix buggy lines in Verilog by suggesting replacement codes. However, this approach requires assistance from designers to identify the bug location. Moreover, the method is limited to evaluation of bug-related functionality.

2) Trojan Generation: LLM is effective for structural generation of digital Trojans with different trigger types [19]. In [23], LLMs have been leveraged to identify specific segments in Verilog codes that can be maliciously modified as part of a Trojan attack. For example, an advanced encryption standard (AES) module can be exploited to manipulate key generation, or branch prediction logic within a CPU operating in privilege mode. In [24], the LLM is guided by custom prompts to insert specific vulnerabilities such as deadlock states into finite-state machines.

3) Trojan Detection: Saha et al. [24] present a vulnerability detection technique based on prompt engineering. However, the current state-of-the-art methods [20], [24] focus exclusively on detecting vulnerabilities in digital circuits. In contrast, [1] is the first method to use LLMs for detecting and localizing analog Trojans within circuit netlists; this problem has not been explored by prior LLM-aided techniques. This article also introduces the first LLM-based method for detecting syntactical bugs in analog circuit netlists.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

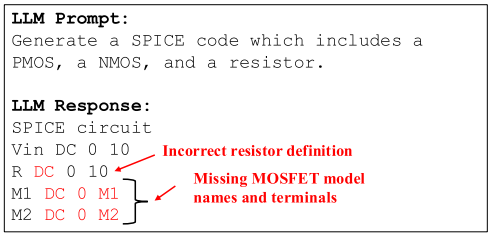


Fig. 1. Examples of SPICE syntax errors generated by the Llama-2 model (highlighted in red).

4) Assertion Generation: In [22], LLM prompts are curated to produce assertions. Next, these assertions are passed through a simulator to verify the correctness. However, designer expertise is needed for assessing these security assertions.

C. Nature of Analog Trojans

Analog Trojans are stealthy modifications in the A/MS circuit, either in the design or fabrication stage. Trojans consist of two main components: trigger and payload. The trigger is a specific, rare condition (such as input voltage range) where the Trojan is activated, and it remains dormant otherwise. The trigger circuit can be conditionally gated with an AND or OR gate to activate the Trojan only in specific input voltages. The payload modifies the circuit functionality, such as disrupting the primary output voltage, and causes performance degradation. The payload is activated by a detector circuit that monitors the charge buildup of the capacitive component of the Trojan circuit and activates the payload when the capacitor voltage reaches a threshold. Recent work shows the impact of an analog Trojan, namely, A2 [5], which can be stealthily inserted during the design and fabrication phases of an analog design. Due to its small footprint, it can be maliciously inserted in unused parts during design phase. The trigger for the A2 Trojan is software-controlled, i.e., the trigger is activated when a rare instruction is executed. As an extension to the A2 Trojan, Gupta et al. [6] implement the DELTA Trojan, which uses a glitch generator for the trigger circuit, and can be inserted in any net of a circuit irrespective of it being rarely activated.

In [9], it has been shown that A2 Trojans remain mostly dormant due to their insertion in the less sensitive paths of a circuit. Upon their activation, the primary output voltage behavior is impacted, leading to significant performance degradation. Therefore, it is necessary to detect and localize these malicious circuits before the netlist is sent to fabrication.

D. Prior Work on Analog Trojan Detection

In [7], a current-sensing-based circuit is used to detect analog Trojans such as A2 at run-time. In [9], analog neural twins are used to identify the least sensitive paths in an analog circuit through sensitivity analysis. After identifying critical paths, circuit watermarks are inserted to make these paths observable at the circuit output. This ensures that even if the Trojan is stealthy and its effect is not captured at the primary output of the circuit, it can still be detected by the watermarks.

Although Chaudhuri et al. [9] effectively identify all the Trojan hotspots, there are two significant limitations: 1) the area overhead associated with added watermarks increases with the complexity of the analog design and 2) the specific Trojan-inserted nodes are not identified, i.e., localization of the detected Trojans is not performed; this limitation makes it difficult to pinpoint the exact nodes affected by the Trojans, making targeted mitigation infeasible. These limitations motivate the exploration of specialized techniques that leverage the contextual ability of LLMs. LLMs are known for interpreting and generating texts and are capable of interpreting contexts across various programming languages such as C, C++, and Python. A significant portion of the training data for LLMs is sourced from Github repositories. LLMs such as Llama-2 and Llama-3 are primarily trained on Python datasets [25]. In the realm of digital design, the availability of Verilog code in training datasets is relatively limited compared with other programming languages [13]. The scarcity is even more for analog design data, particularly for SPICE, which has less code available in open-source repositories. This poses a challenge for LLMs such as Llama-2 to understand SPICE syntax comprehensively; a preliminary evaluation using basic SPICE netlist examples shows that these models struggle to understand SPICE syntax and exhibit limited understanding of the connection between components such as nMOS, pMOS, resistors, and capacitors. For instance, correct assignment of drain, gate, and source terminals is frequently misclassified as functional errors. In addition, the LLMs frequently generate incorrect representations of MOSFETs within the SPICE netlist. Fig. 1 illustrates the common errors generated by the LLM. Despite these challenges, GPT-3.5 shows the capability of understanding basic SPICE syntax and type of circuit configuration being implemented in a SPICE netlist.

E. Motivation

Prior LLM-based techniques have primarily focused on inserting and detecting vulnerabilities in the digital domain [19], [20], [24]. Moreover, no mitigation strategy was explored in these works. In contrast, SPICED+ is designed to diagnose and mitigate stealthy analog Trojans. Integrating SPICED+ with LLM-guided evaluation offers several key advantages.

1) *Textual Analysis Capability:* By leveraging textual analysis capability, LLMs can effectively parse simulation log files without any manual intervention, thereby offering large productivity boost in terms of reduced engineering hours and log processing runtime. LLMs understand the tabular structure of logged voltage and current values, eliminating the need for explicit conversions to graph structures.
2) *Netlist Identification:* Pretrained LLMs can understand the basic syntax and structure of SPICE netlists.
3) *Simplified Circuit-to-Graph Conversion:* While supervised learning models and heuristic-based strategies typically require a graph-based representation of the circuit, LLM bypasses this step. Through pretraining and few-shot learning, LLMs understand SPICE syntax and learn the rules of SPICE connectivity, saving preprocessing effort.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

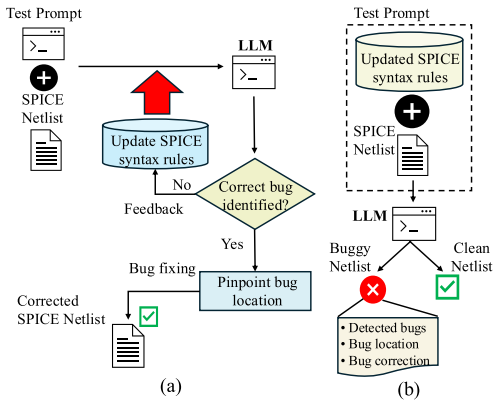


Fig. 2. (a) Refinement of syntax rules by instruction-following approach. (b) Identifying and mitigating bugs in a SPICE netlist (adapted from [1] and redrawn).

4) *SPICE Simulations and Implicit Learning of Trends:* Simulations are carried out through SPICE, which removes inaccuracy in simulation. Interpretation of simulation results for identifying anomalous nodes involves converting them into datasets and then performing statistical analysis on them. While statistical analysis is more accurately done using numerical methods, LLMs try to implicitly learn normal trends in nodal data versus abnormal via their internal attention computation between tokens, where the tokens represent the data entries from voltage and current simulations.
5) *Trojan Detection and Localization:* Equipped with the ability to parse logs, analyze netlists, and identify outlier patterns, LLMs can potentially identify Trojan-affected nodes whose voltage and current profiles manifest as anomalous.
6) *No Area and Power Overhead:* Leveraging the computational abilities of LLMs obviates the need for any additional Trojan detection hardware in the circuit.

F. Threat Model

In this work, we address two critical threats in A/MS designs which are often introduced by third-party vendors or untrusted foundries. The first threat arises from syntactical bugs, which are unintentionally introduced during the intellectual property (IP) design process. These bugs can be connectivity errors (incorrect connections between transistor components) or floating inputs of transistor gates. These bugs can compromise the circuit functionality if left unnoticed during the design phase. The second threat involves the risk of malicious Trojans, which are intentionally inserted into the netlist during the design phase. These Trojans are carefully crafted to remain dormant under normal operational condition and are activated only under specific trigger conditions. Once triggered, they can cause performance degradation and circuit malfunction (e.g., leak-sensitive data, increase power consumption).

III. LLM for Syntactical Bug Detection

A. Proposed Workflow

Fig. 2 illustrates the proposed flow of guiding the LLM to detect a wide range of syntactical bugs while reducing the likelihood of incorrectly flagging correct lines as syntax errors. Note that this method does not involve any LLM training or fine-tuning; instead, it focuses on improving the bug detection capability of LLM through precisely structured prompts and refinement of SPICE syntax rules based on real-time feedback.

1) Refinement of Syntax Rules: Pretrained LLMs such as GPT-3.5 and GPT-4 often flag syntactically correct SPICE code as buggy (false positives) if they are not provided with explicit SPICE syntax rules. This leads to a high false-positive rate. To address this, we implement a feedback-based refinement approach to manually update the SPICE syntax rules after each LLM response. This process is repeated until a minimum number of iterations, $n_{\max}$, are reached, which reduces the false positives while maintaining bug detection accuracy. The feedback-based approach is shown in Fig. 2(a). The steps for obtaining $n_{\max}$ are as follows.

1) We begin with a baseline set of SPICE syntax rules and apply them to a bug-inserted netlist in the LLM prompt.
2) After each iteration, we check whether the LLM correctly identifies the bug and its location. If the LLM incorrectly flags a syntactically correct line as buggy, we manually update and refine the syntax rules in the prompt accordingly.
3) The updated rules are then applied to the same netlist. The process is repeated until the number of false positives stabilizes, ensuring that further iterations do not improve the accuracy significantly.

2) Evaluation on Test Dataset: After the refinement of SPICE syntax rules, we apply them for evaluation on a dataset of SPICE netlists [shown in Fig. 2(b)] and we prompt the LLM to generate structured bug detection reports. The report includes the following essential components.

1) *List of All Syntactical Bugs:* The LLM compiles a list of all the detected bugs, which includes connection error (missing connections or floating nodes), insertion error (unintended or intended insertion of additional transistor components), or incorrectly defined parameters.
2) *Location of the Bugs in the Netlist:* The LLM pinpoints the exact lines in the SPICE netlist where each bug occurs and specifies the associated component or connection name.
3) *Suggestions for Correction:* For each detected bug, the LLM provides recommended actions to correct the bugs and generate a revised, error-free netlist. Fig. 3] shows an example prompt incorporating feedback-based refinement of syntax rules for improved bug detection accuracy.

B. Advantages of LLMs Over Static Analyzers for Syntactical Bug Detection

Static analysis is widely used for detecting both software and hardware bugs. In [26] and [27], static analyzers are incorporated to detect real-world software bugs based on a predefined set of rules. While the static analyzer performs rule-based checks, it lacks the ability to generalize on unknown syntactical bug patterns that fall outside its scope. In contrast, LLMs excel at dynamically updating to new rules without requiring any programming logic updates. A key strength

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

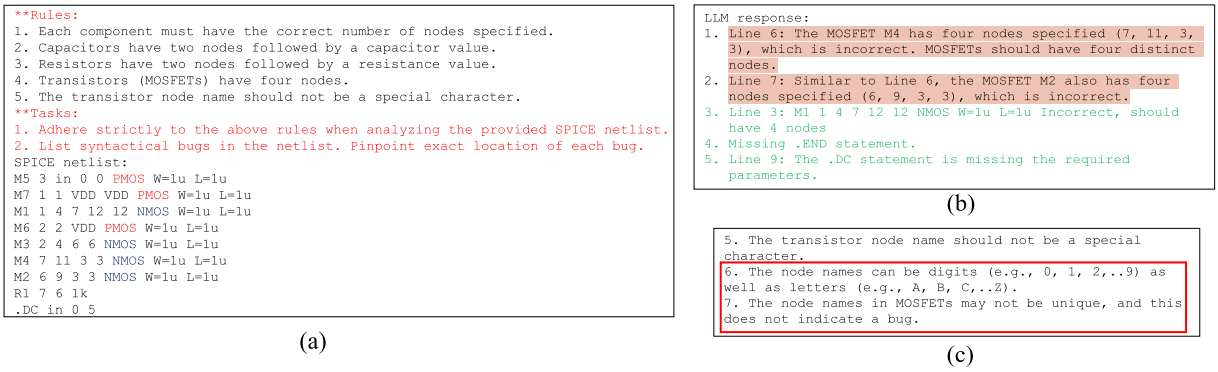


Fig. 3. (a) Example prompt highlighting the SPICE syntax rules for syntactical bug detection and localization. (b) LLM correctly detects the bugs injected in the netlist; however, it incorrectly flags Lines 6 and 7 as bugs (highlighted in red). (c) Explicitly updating the rules in the prompt to reduce false positives [1].

of LLMs in our work is its ability to analyze contextual relationships between components and iteratively refine the SPICE syntax rules based on real-time feedback. When a bug is missed, LLMs can be guided using additional examples and rules, which requires minimal manual effort. This iterative loop is not present in the static analyzer, which requires manual code changes and reprogramming when a bug is missed.

IV. Supervised-Learning-Based Framework Using LLM for Trojan Detection

A. Proposed Trojan Detection and Localization Framework

While Section III focuses on identifying and correcting syntactical bugs in the SPICE netlist, this section addresses functional bug (analog Trojan) detection. A Trojan-impacted node is defined as one of the circuit nodes where the Trojan is inserted. Based on prior work on analog Trojan detection [7], [9], we make the following observations regarding the current and voltage behavior of Trojan-impacted nodes.

1) *Deviation in Primary Output Voltage:* As observed in [9], when a Trojan is inserted into one of the sensitive paths of an analog design, its impact is captured at the primary output voltage. Specifically, the output voltage exceeds the desired specifications when the Trojan is activated.
2) *Anomalous Deviation in Circuit Intermediate Nodes:* Nodes impacted by the Trojan exhibit significant deviations in voltage when the Trojan is activated. This deviation can serve as a critical indicator of the presence of a Trojan.
3) *Anomalous Surge in MOSFET Current:* According to [7], upon Trojan activation, some MOSFETs within the analog design draw a substantially higher current. This current behavior deviates significantly from the normal current behavior observed when the Trojan circuit is dormant.

The supervised learning procedure [shown in Fig. 4(a)] is adapted for the LLM as shown in Fig. 4(b). We link each Trojan characteristic to a supervised learning rule as follows.

1) *Deviation in Primary Output Voltage (Supervised Learning Rule 1):* We craft prompts that highlight output voltages that exceed the desired circuit specifications. Based on these output voltages, we ask the LLM to identify the corresponding input voltages. These input voltages are labeled as "Trojan-Activation Inputs," while inputs where the Trojan is inactive are labeled as "Normal Inputs."
2) *Anomalous Deviation in Circuit Intermediate Nodes (Supervised Learning Rule 2):* Fig. 5 shows the voltage deviations in intermediate nodes of a Trojan-inserted circuit across different input voltages and how their voltage deviations compare across nodes for anomaly detection. The LLM identifies the nodes that exhibit: a) a significant voltage deviation between "Trojan-Activation Input" and "Normal Input" ranges and b) highest deviation among all the intermediate nodes for each "Trojan-Activation Input." The LLM applies one of the following rules to determine the Trojan-impacted nodes.
   a) *Rule #1:* Union of nodes identified in Steps (a) and (b).
   b) *Rule #2:* Intersection of nodes obtained in Steps (a) and (b).
3) *Anomalous Surge of MOSFET Current (Supervised Learning Rule 3):* We teach the LLM to examine current simulation logs and identify transistors that generate unusual current spikes under "Trojan-Activation Inputs" compared with "Normal Inputs." The nodes corresponding to these transistors are flagged as suspect nodes.

The supervised learning rules are defined at the beginning of the prompt, followed by few-shot examples, where it sees a limited number of labeled netlists ("Trojan" or "Trojan-Free") along with explanations based on these rules. These explanations highlight deviations in current and voltage behavior that indicate potential Trojan activity in the nodes. By combining rules with few-shot examples, the LLM learns to distinguish Trojan behavior from normal circuit behavior, thereby significantly reducing false positives. For example, the LLM detects deviations in transistor current across input voltages (based on *Supervised Learning Rule 3*). Similarly, by analyzing the voltage deviations in the nodes, LLMs can detect which nodes deviate abruptly from their normal behavior (based on *Supervised Learning Rules 1 and 2*).

B. Prompt Design and Reducing Hallucinations

We acknowledge that hallucinations—where LLM generates incorrect outputs—were encountered during the experiments.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

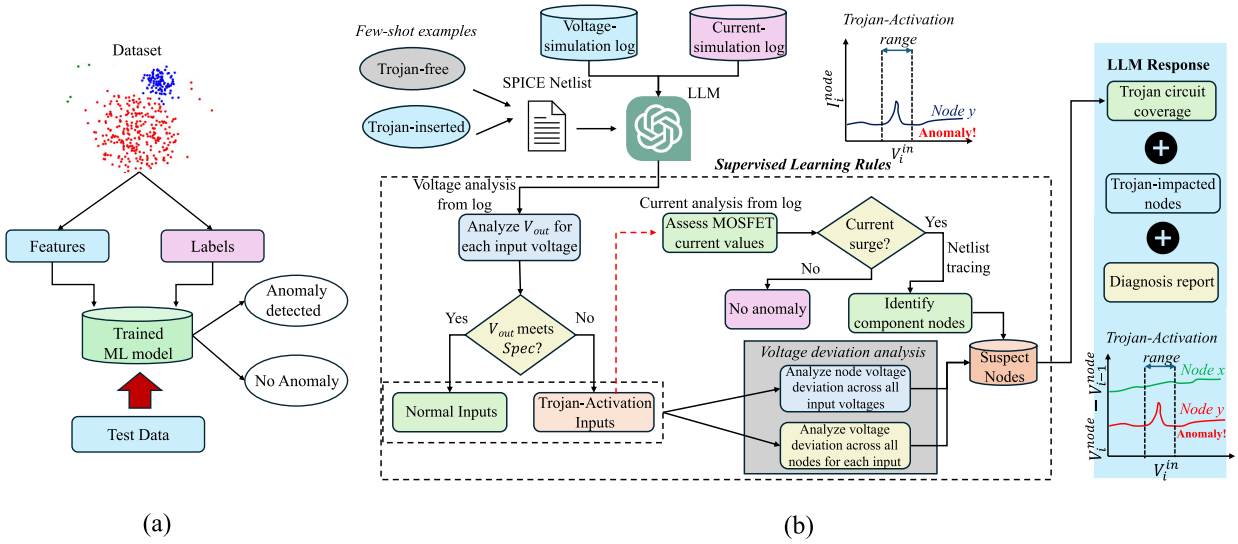


Fig. 4. (a) Flow of supervised learning approach using ML models. (b) **SPICED** workflow: Using the supervised learning analogy to locate Trojan-impacted nodes using LLM (V_{out}: primary output voltage, Spec: desired output voltage specifications, V_i^{node}(I_i^{node}): node voltage (current) corresponding to the ith input voltage sample V_i^{in}) [1].

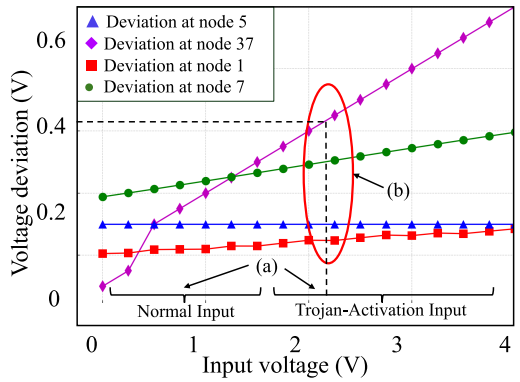


Fig. 5. Voltage deviation analysis for A2 Trojan-inserted netlist of circuit "642" from AMSNet [15]. Voltage deviation of a node x is given by $V_i^x - V_{i-1}^x$, where $i-1$ and i are consecutive input voltage samples. (a) Analyze voltage deviation of each node across the "Normal Input" and "Trojan-Activation Input" ranges. (b) Analyze voltage deviation across nodes in the "Trojan-Activation Input" range. Combining (a) and (b), we observe that node 37 is a Trojan-impacted node [1].

This happened when we provided long-context prompts to the LLM and did not explicitly define the rules for identifying Trojan behavior. To address hallucinations and improve the reliability of our framework, we adopted the following steps.

1) *Clear, Well-Defined Supervised Learning Rules:* Through several experiments, we observed that providing Trojan detection rules separately for voltage and current deviations significantly improved the LLM's ability to identify the suspect nodes. When hallucinations occurred, we systematically clarified the supervised learning rules based on the model's response, ensuring that the rules are precise and unbiased.
2) *Few-Shot Examples:* We provided few-shot examples including both Trojan-free and Trojan-inserted netlists as prompts to the LLM. This approach helped the LLM learn the correlation between node voltage deviations and anomalous behavior, guided by supervised learning rules.
3) *Shortening Contexts:* Since the netlist, current simulation log, and voltage simulation log are all fed as inputs to the LLM, the size of these inputs highly impacts the context window limit. Given the context size limitation, we developed a script to parse the HSPICE simulation logs to remove the redundant information and retain the node voltage/current data only. This significantly reduced the size of the simulation logs, allowing all the inputs to fit within the context window, including the supervised learning rules.

C. Comparison to Supervised ML

Supervised machine learning (ML) models have been used extensively in various Trojan detection tasks, mostly in the digital domain [28], [29]. In the context of analog Trojan detection, the ML model has to be trained on a large dataset of node voltage and current behavior specific to an analog design. However, the model has to retrained for a new design, which requires significant preprocessing and training overheads. Moreover, without additional stages including multilabel classification, they lack the ability to perform Trojan diagnosis or mitigation.

In contrast, LLMs are inherently generalizable and flexible; by including the desired output specifications of a design directly in the input prompt, LLMs can adapt to new circuits without any retraining. Moreover, LLMs bypass any preprocessing steps due to their inherent textual analysis capability. While ML models require a time-intensive training data preparation procedure (parsing SPICE simulation logs to extract node voltage/current and data labeling), LLMs can directly analyze simulation logs and extract the relevant information as guided by the input prompt. This makes LLMs versatile, leading to quick deployment in diverse analog designs.

V. SPICED+: Automated Trojan Mitigation Framework Using Iterative Correction

SPICED links different characteristics of stealthy, analog Trojans with the corresponding supervised learning rules to improve Trojan coverage. However, several evaluated benchmark circuits report significant false negatives. A false negative

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON BETWEEN SPICED AND SPICED+

Parameters	SPICED	SPICED+
Objective	Detect Trojan-impacted nodes	Fine-grained Trojan detection and mitigation
Analysis method	Rule-based	SPICED+Iterative correction
Training needed?	✗	✗
Handling false positives	✗	✓
Confidence score	Not considered	Used to prioritize suspect nodes
Diagnosis depth	Simulation logs	Simulation logs+ suspect nodes
Trojan coverage	Medium	High
Trojan mitigation	✗	✓

occurs when a Trojan line is incorrectly flagged as benign. In general, it is highly crucial to correctly identify all the Trojan lines to enhance the effectiveness and reliability of the framework. In this section, we address the following questions.

1) Can we formulate a confidence scoring metric that quantifies the severity of a particular node to be Trojan-impacted?
2) How can LLMs-as-a-service be used for Trojan mitigation by applying deep understanding of SPICE netlists?

Table II presents a detailed analysis of SPICED+ and highlights the benefits of using an iterative correction-based workflow for refined analysis of SPICE netlists.

A. Evaluation of Confidence Scores for Suspect Nodes

The confidence score serves as a critical metric in determining the likelihood that a suspect node has been impacted by a Trojan. Using the confidence score, we ensure that nodes with the highest deviation from normal behavior are analyzed in greater depth. Based on Supervised Learning Rules 1 and 2 (shown in Section IV), we compute the average voltage for node n_i across all "Trojan-Activation Inputs" as

$$V_{\text{TA}}(n_i) = \frac{1}{J}\sum_{j=1}^{J} V_{\text{TA},j}(n_i). \tag{1}$$

Similarly, we average the node voltages for "Normal Inputs"

$$V_N(n_i) = \frac{1}{K}\sum_{k=1}^{K} V_{N,k}(n_i). \tag{2}$$

Using (1) and (2), we calculate the deviation $D(n_i)$ for node n_i as: $D(n_i) = |V_{\text{TA}}(n_i) - V_N(n_i)|$, where $D(n_i)$ is the absolute deviation between the node's behavior under "Trojan-Activation Inputs" and "Normal Inputs."

Finally, the confidence score for node n_i is calculated as

$$\text{CS}(n_i) = \frac{D(n_i)}{D_{\max}}. \tag{3}$$

In (3), $D_{\max}$ indicates the range between the upper and lower bounds of the suspect node voltage. Each suspect node is assigned a confidence score between 0 and 1, representing the degree of confidence in its classification.

We embedded (3) in the LLM prompt and performed extensive experiments to determine a predefined threshold for the confidence score, beyond which a node would be flagged as Trojan-impacted. Specifically, we prompted the LLM to generate the confidence score of several benign nodes (where a Trojan is not inserted). Based on these experiments, we observed that the LLM consistently generated very low confidence scores for the benign nodes, typically below 0.2. In contrast, for actual Trojan-impacted nodes correctly identified by the LLM, the confidence scores were observed to be significantly higher, exceeding 0.7. Based on these observations, we established a classification rule, where confidence score greater than 0.5 is classified as Trojan-impacted, and confidence score less than 0.5 is classified as normal.

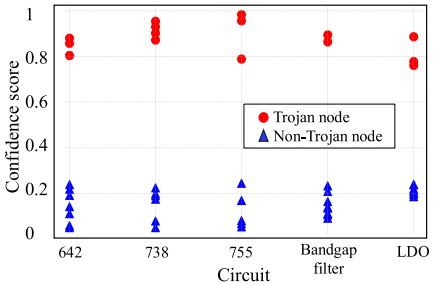


Fig. 6. Scatter plot of LLM-generated confidence scores [based on (3)] of Trojan-impacted and non-Trojan nodes.

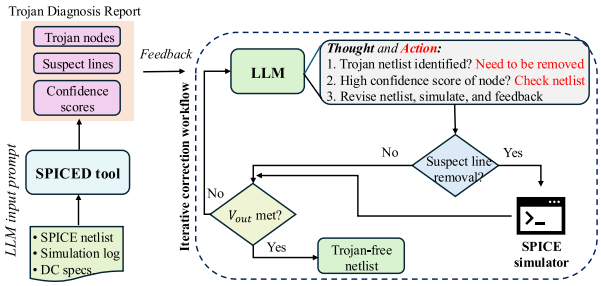


Fig. 7. Combining user-defined decisions and LLM analysis for the iterative correction of a Trojan-inserted SPICE netlist.

We evaluated the effectiveness of this confidence scoring metric on the following benchmark circuits: 642, 738, and 755 from the AMSNet dataset [15], bandgap filter, and LDO [30]. The results are shown in Fig. 6; we observe a clear distinction between the two node classes, indicating the accuracy of the confidence scoring approach.

B. SPICED+: Leveraging LLMs for Iterative Correction of SPICE Netlists

In scenarios where a list of Trojan-impacted nodes and Trojan lines are generated by SPICED, the absence of verification can lead to false negatives, which impacts the detection accuracy. To address this, we extend SPICED to SPICED+, an automated tool for Trojan detection and mitigation that conducts fine-grained analysis of suspect nodes and Trojan lines identified by SPICED. Fig. 7 illustrates the proposed Trojan correction workflow. Table II presents the enhancements made to our existing LLM-based detection framework. The key elements of SPICED+ are as follows.

1) *Netlist Parsing and Analyzing Simulation Logs:* SPICED+ successfully parses the SPICE netlist and the simulation logs generated by HSPICE. It extracts relevant information such as voltage and current values and compares them against predefined specifications.
2) *Incorporating Multiple Tools:* SPICED+ integrates both SPICED and HSPICE in the workflow. While SPICED is used for suspect line identification and confidence score

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

Input: SPICE netlist N, Simulation log files Sim_N, Performance spec V_{out}
Output: Revised Trojan-free netlist N^{tf}
Tools: SPICED, HSPICE
Thought: I will read Sim_N and check if it is Trojan-free
Action: Analyze Sim_N to check if specs are met.
if V_{out} *not met* **then**
  Run **SPICED**: Save initial netlist, simulation results, suspect lines as baseline $State_0 = (N, Sim_N, L_{sus})$
  Thought: I have received a netlist with suspect lines. I will begin evaluating each suspect line.
  for *suspect line* l *in* L_{sus} **do**
    Thought: I will remove this suspect line.
    Action: Remove l from N to obtain N_i and run HSPICE.
    Thought: Let me compare Sim_N with Sim_{N_i} to check if performance specs are met.
    Action: Compare the simulation logs.
    Thought: I will now evaluate the results to decide if the line removed was part of the Trojan.
    if ***llm_check****(Sim_{N_i} matches V_{out} and no anomalous Trojan behavior)* **then**
      Thought: The correct line has been removed, and the Trojan behavior is neutralized. I can stop now.
      Action: Set final netlist and stop. Set $N^{tf} = N_i$ and **stop exploration**
    end
    if ***llm_check****($Sim_{N_i} \sim Sim_N$ and V_{out} not met)* **then**
      Thought: I removed the correct line, but the Trojan is still present. I should continue exploring.
      Action: Move to the next suspect line.
    end
    if ***llm_check****(Sim_{N_i} degrades and V_{out} not met)* **then**
      Thought: I removed the wrong line, and the circuit behavior degraded without mitigating the Trojan. I'll undo this change.
      Action: Restore previous state and explore the next suspect line.
    end
    Save current state $State_i = (N_i, Sim_{N_i})$
  end
end
Thought: I have successfully identified and removed the Trojan lines. The final netlist is ready.
Action: Output revised netlist without Trojan lines. Return N^{tf}

Fig. 8. Pseudocode for SPICED+ framework.

generation, HSPICE is used for resimulation of updated netlists and validation of performance specifications.

3) *Iterative Line Removal and Resimulation:* SPICED+ performs iterative exploration by removing suspect lines from the netlist one at a time, followed by rerunning the simulation of the modified netlist. The simulation results are compared with the previous log to detect performance improvements or degradations.
4) *State Management for Comparison:* A key feature of SPICED+ is to store the netlist state and the corresponding simulation log at various stages of the iterative correction process. This allows for rollback in case removing a suspect line adversely impacts the circuit performance.

The motivation for this algorithm is to identify and mitigate Trojans that had been inserted in the SPICE netlist. Mitigation, in this context, refers to removing the Trojan lines from the circuit netlist. We achieve this by iteratively modifying the netlist based on predefined thought and actions and leveraging LLM capabilities for analyzing specific conditions for confirming Trojan presence. The pseudocode for this procedure is shown in Fig. 8. The process begins with an initial analysis of the simulation log Sim_N to determine whether the circuit specification V_{out} is met. If the specification is not met, this indicates the presence of a Trojan in the netlist. This is followed by a two-stage workflow of SPICED+.

In Stage 1, we use SPICED to identify suspect lines L_{sus} and Trojan-impacted nodes in the netlist. Stage 2 is a user-guided evaluation, where each suspect line is systematically analyzed to check whether it is part of a Trojan circuit. The ***llm_check*** indicates that the LLM verifies whether the simulation log satisfies the circuit specifications, without requiring manual intervention. It also determines whether the confidence score of the nodes within each suspect line indicates a potential Trojan-impacted node. Based on the output of the ***llm_check***, one of the following **Actions** are taken.

1) Remove the suspect line from the netlist and resimulate to obtain the updated simulation logs.
2) Undo the removal of the suspect line and restore the netlist and simulation log of the previous state.
3) Stop the iterative-correction procedure once all Trojan lines are correctly identified and mitigated.

The decisions associated with these checks are collectively referred as **thought**. Both **thought** and **action** represent predefined rules that instruct the LLM on how to proceed based on different conditions during the iterative analysis process.

The entire process of ***llm_check*** and the subsequent actions are seamlessly automated using Python scripts. This ensures that Trojan lines are mitigated efficiently, without requiring the user to manually review the simulation log or the netlist.

Handling False Negatives: In Stage 1, there might be actual Trojan lines that remain undetected (and not subsequently processed in Stage 2). The Trojan lines identified in Stage 1 are iteratively mitigated in Stage 2, and the output voltage is compared against the desired specifications. If the specifications are not met even after all identified Trojan lines are removed, this indicates the presence of undetected Trojan components or false negatives. In such cases, we rerun the two-stage workflow until the output specifications are satisfied. Although this process may require additional iterations, the computational overhead is justified given the necessity of ensuring security in industry-grade, safety-critical applications.

VI. Experimental Results

A. Experimental Setup

For our experiments, we evaluate the circuits 642, 738, and 755 from AMSNet [15] and Github [30]. These circuits include differential amplifier, inverter, OPAMP, and bandpass filter, thus providing a comprehensive dataset for LLM evaluation. Table III lists the number of components in each evaluated netlist. For syntactical bug analysis, we have compiled a total of 18 syntactical bugs that are mentioned in [31]. We assign a difficulty level (easy, medium, and difficult) to each bug for testing the effectiveness of LLM in identifying bug-inserted SPICE netlists. The bug benchmark used for LLM evaluation is shown in Table IV. The benchmark includes four easy, six medium, and eight difficult syntactical bugs.

For the Trojan detection experiments, we evaluate the well-known A2 analog Trojan [5]. We embed A2 into the SPICE netlist of the evaluated benchmark circuits to generate the corresponding Trojan-inserted netlists. The Trojan is considered to be internally triggered by an intermediate node of the

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

TABLE III
NUMBER OF COMPONENTS IN THE EVALUATED BENCHMARK CIRCUITS

Circuit	#Nodes	#Transistors
Common source amplifier	4	2
Inverter	5	2
Current mirror	6	4
Differential amplifier	8	5
642 [15]	17	10
738 [15]	15	10
755 [15]	19	13
OPAMP	14	10
Bandgap filter	265	44
LDO	1655	347

TABLE IV
LIST OF SYNTACTICAL BUGS WITH VARYING DIFFICULTY LEVELS FOR EVALUATING LLM PERFORMANCE

Description of Bug	Example
Missing node of transistor	M1 2 3 0 PMOS
Missing .END statement	Netlist without terminating .END
Missing transistor model	M20 2 3 0 0 (model undefined)
Floating node	in
Extra node in transistor	M20 2 3 2 0 **4** PMOS
Incorrect resistor format	R1 in out **1Ki** (should be 1k)
Incorrect subcircuit definition	.SUBCKT a b (instance name undefined)
Missing capacitor value	C12 in out
Incorrect usage of transient analysis	.tr 100p (simulation duration undefined)
Missing voltage value	VB 5 0
Special characters in node names	M**!2** 3 5 0 0 NMOS
Incorrect .PRINT statement	.PRINT TRAN **in** (should be **V(in)**)
Incorrect current source definition	Ib 1 0 1M
Incorrect transistor name	M1 2 3 0 0 **NMOSC**
Missing .END in subcircuit	Subcircuit without .ENDS
Incorrect parameter definition	.PARAM **R1=**
Incorrect .OPTIONS definition	**.OPTION** POST (should be .OPTIONS)
Missing value in .DC	.DC vin 0.1 5 (missing increment value)

The syntactical bugs in the examples are indicated in **bold**. The bug difficulty levels are color-coded as green (easy), yellow (medium), and red (difficult).

TABLE V
INFORMATION PROVIDED TO THE LLM FOR SYNTACTICAL BUG AND TROJAN DETECTION TASKS

Task	Information provided
Bug detection	SPICE netlist + syntax rules
Trojan detection	SPICE netlist + corresponding simulation log files (current and voltage) + circuit specification

analog design, thus emulating a realistic attack scenario shown in [32]. A schematic of an internally triggered A2 Trojan is shown in Fig. 9. Table V lists the information available to the LLM for performing bug and Trojan detection tasks.

We use GPT-3.5-turbo-16k and GPT-4o-mini API models for evaluation. The experiments are carried out on a 3-GHz AMD EPYC 7313 CPU with 2 TB of RAM. Although we have demonstrated SPICED+ using specific LLMs, our method is model-agnostic and can be incorporated using other LLMs as well. However, note that previous LLMs (such as Llama-2) are limited in their capacity to process SPICE syntax and often generate incorrect component representations (shown in Fig. 1). Therefore, it is critical to incorporate advanced LLMs that can learn the topological relationships among circuit nodes and infer detailed node behaviors for anomaly detection tasks.

B. Evaluation Metrics

We use the following metrics to evaluate the effectiveness of the LLM for syntactical bug and Trojan detection.

1) *Bug Coverage (%):* This metric represents the percentage of syntactical bugs detected by the LLM out of the total number of bugs embedded within a SPICE netlist.
2) *Bug Resolved (%):* It represents the percentage of detected syntactical bugs that are correctly fixed by LLM.

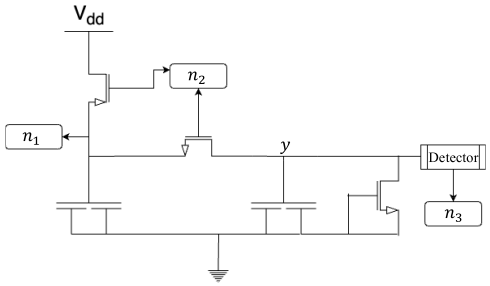


Fig. 9. Schematic of an internally triggered A2 Trojan: n_1, n_2, and n_3 represent the input, trigger, and output nodes of the Trojan in an analog design, respectively. These nodes are impacted by the Trojan.

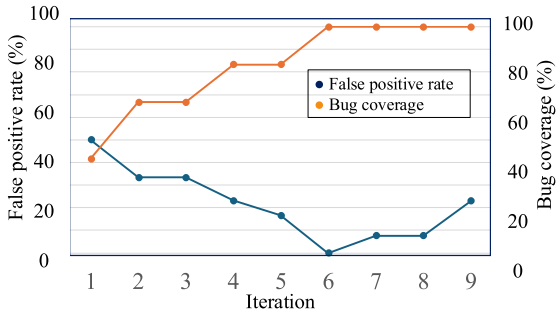


Fig. 10. Determining $n_{\max}$ for feedback-based refinement of SPICE syntax rules.

3) *Trojan Identified:* It indicates whether the LLM has correctly detected at least one Trojan component in the netlist.
4) *Trojan Coverage (%):* It is the percentage of Trojan lines that are correctly identified by the LLM.
5) *Recall or True Positive Rate (%):* It is defined as $\text{TPR} = (\text{TP}/(\text{TP}+\text{FN})) \times 100$, where TP indicates the number of Trojan nodes that are correctly identified and FN is the number of Trojan nodes missed by the LLM.
6) *FPR (%):* It is the percentage of non-Trojan nodes that are incorrectly flagged as malicious.
7) *Precision (%):* It is the percentage of correctly predicted Trojan-impacted nodes out of all the predicted impacted nodes. It is denoted by $\text{Precision} = (\text{TP}/(\text{TP}+\text{FP})) \times 100$, where FP indicates the number of non-Trojan nodes that are incorrectly classified as Trojan-impacted by the LLM.
8) *FPR_{tr} (%):* It is the percentage of non-Trojan lines that are incorrectly classified as malicious by the LLM.
9) *n_{spec}:* Number of iterations needed by SPICED+ (including the steps of removing suspect lines, undoing changes, and rerunning SPICED) to ensure correct circuit operation.
10) *TMR (%):* Trojan mitigation rate or TMR is the percentage of Trojan lines that SPICED+ successfully removes from the netlist. It is formulated as: $\text{TMR} = (N_{\text{rem}}/N_{\text{tr}}) \times 100$, where N_{tr} is the number of correctly detected Trojan lines and N_{rem} is the number of Trojan lines removed from the netlist.

C. Syntactical Bug Detection

Determining n_{max}: We use the common source amplifier as the test netlist for evaluation. The results are shown in Fig. 10.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCE OF LLMS IN IDENTIFYING SYNTACTICAL BUGS ACROSS DIFFERENT CIRCUIT TYPES

Case	Circuit Type	Bugs inserted			Bugs Detected (%)			Bug Coverage (%)	FPR (%)		Bugs resolved (%)
		Easy	Medium	Difficult	Easy	Medium	Difficult		Without Instructions	**With Instructions**	
1	CS amplifier (R load)	2	5	5	100	100	100	100	38.8	0	100
2	CS amplifier (RC load)	3	4	6	100	75	66.7	76.9	28.5	7.1	90
3	NMOS Transistor	4	5	6	100	100	83.3	93.3	16.6	0	100
4	Switched capacitor	4	4	7	100	100	71.4	86.6	23.5	5.8	100
5	Inverter	4	4	7	100	75	71.4	80	14.2	0	83.3
6	Differential amplifier	3	6	8	100	100	87.5	94.1	15.7	0	81.2
7	Current mirror	4	6	8	100	83.33	75	83.3	21	6.25	86.6
8	Current mirror (Differential pair)	3	5	7	100	100	71.4	86.6	18.75	0	76.9
9	OPAMP	4	5	7	100	100	100	100	31.2	6.25	87.5
10	Bandgap filter	3	6	5	100	100	80	92.8	31.5	13.3	92.3

Using a refined set of SPICE syntax rules results in a significantly lower FPR compared to the scenario when a basic set of syntax rules is prompted to the LLM.

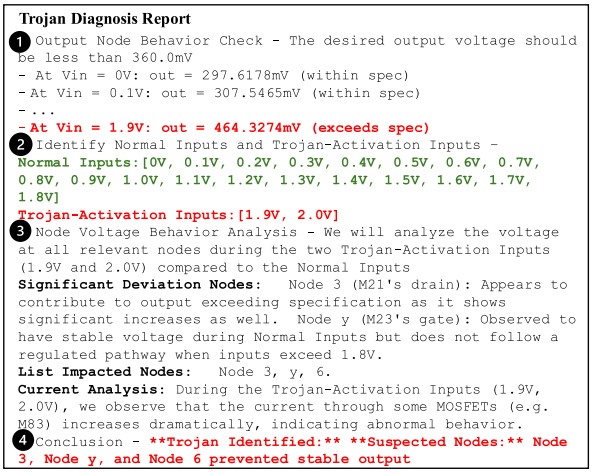


Fig. 11. Diagnosis report generated by SPICED: ❶ From the voltage simulation log, LLM analyzes the output voltage behavior and identifies which input voltages generate out-of-spec output voltage. ❷ Based on the reasoning capability taught to the LLM, it identifies the "Normal Inputs" (highlighted in green) and "Trojan-Activation Inputs" (highlighted in red). ❸ From the labeled "Trojan-Activation Inputs," LLM performs voltage and current analysis as well as provides detailed explanations to identify the Trojan-impacted nodes. ❹ SPICED correctly identifies the Trojan-impacted nodes.

With every iteration, the syntax rules are updated and provided to the LLM through input prompts. This significantly improves the overall bug coverage while reducing the FPR. We select $n_{\max} = 6$, as further iterations result in negligible improvement in performance metrics.

The bug detection results are shown in Table VI. We observe that GPT-3.5 demonstrates higher accuracy and broader bug coverage across all the evaluated SPICE netlists. In addition, the feedback-driven syntax refinement process results in fewer false positives compared with the scenario when SPICE syntax rules were not explicitly provided in the prompt.

D. Evaluation of SPICED and SPICED+

1) Trojan Detection and Localization: To evaluate the effectiveness of SPICED in Trojan detection and localization, we consider the following four cases.

1) *FS:* We provide only few-shot examples to the LLM.
2) *R:* We provide only supervised learning rules to the LLM.
3) R_{FS}*:* We provide both supervised learning rules and few-shot examples, ensuring that LLM correlates the examples with the rules to determine the Trojan-impacted nodes for an unknown netlist.
4) R^*_{FS}*:* We incorporate Rule #2 instead of Rule #1 (Supervised Learning Rule 2), along with other aspects of R_{FS}.

To increase the complexity of the test cases for LLM evaluation, we scrambled the Trojan components and nodes within the netlist and changed the parameters, such as the width-to-length (W/L) ratios of transistors and the capacitor values. The Trojan-inserted netlists used for the experiments are labeled as "netlist_troj_n," where netlist is the benchmark circuit and n indicates the netlist node where the Trojan payload is activated. As shown in Fig. 11, LLM successfully detects the Trojan circuit from a test netlist and also identifies the Trojan-impacted nodes. The evaluation results on the GPT-3.5 model are shown in Table VII. We observe that incorporating supervised learning rules lead to superior Trojan coverage and accuracy of Trojan-impacted nodes compared with scenarios where only few-shot examples are provided to the LLM. An example prompt showing supervised-learning rules and few-shot examples is illustrated in Fig. 12.

Evaluation results of SPICED on GPT-4o-mini are shown in Table VIII. GPT-4o-mini demonstrates a higher accuracy in detecting the Trojan-impacted nodes; contrary to GPT-3.5, GPT-4o-mini is fine-tuned with a broader understanding of anomalies in simulation data and enhanced context retention.

2) Analysis of Confidence Scores: The SPICE netlist and simulation logs corresponding to circuit 642_troj_2 are fed as inputs to the LLM. The LLM response, which includes the confidence scores of suspect nodes, is shown in Fig. 13. SPICED successfully detects all the Trojan-impacted nodes, assigning confidence scores based on the deviation of the node voltages observed between the "Trojan-Activation Inputs" and "Normal Inputs" ranges.

3) Trojan Correction: Table IX shows the effectiveness of SPICED+ in accurately identifying Trojan circuits and successfully mitigating them from analog netlists, outperforming the standard GPT-4o model. After multiple iterations, the false positives decrease and Trojan coverage improves significantly. Within ten iterations, SPICED+ successfully removes all the detected Trojan lines from the SPICE netlist. In some evaluated circuits, the TMR is not 100%; however, the circuit still meets the desired specifications. This is because the transistors in A2, which significantly impact circuit performance, are fully removed. The remaining capacitive components, which are

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

System

```
LLM Prompt:

You are an advanced language model trained to analyze SPICE
netlists and identify anomalies that could indicate the
presence of hardware Trojans. You will be provided with a
SPICE netlist, simulation log, and desired specifications of
the output node. You will check if there are normal behavior
patterns of MOSFETs in the simulation log and use this
knowledge to check if there is a Trojan circuit in the
netlist. Using the supervised learning rules given below,
check if there is a Trojan circuit in the netlist and identify
the Trojan-impacted nodes.")

**Supervised Learning Rules**
// Analyze voltage simulation log
…Obtain suspect list 1
// Analyze current simulation log
…Obtain suspect list 2
```

User (few-shot examples)

```
SPICE Netlist X
//. . .
Voltage Simulation Log Data
//. . .
Current Simulation Log Data
//. . .
Desired Output Specifications
`The voltage at node 'out'
should be between 0V and 1.5V'
```

Example Response

```
Netlist X contains a Trojan.
The Trojan is:
//. . .
List of Trojan-impacted nodes:
//. . .
Detailed Diagnosis Report
//. . .
```

(a)

User

```
Now that you have learned and understood the above
example(s), you have the knowledge to identify a Trojan
circuit in a SPICE netlist. You will be provided with a
new SPICE netlist, its corresponding simulation log
file, and the desired specifications for the primary
output voltage behavior.
Tasks
//Please follow the supervised learning rules to
perform the following tasks:
**Identify the Trojan
**Identify Trojan-impacted nodes
**Provide a diagnosis report explaining the abnormal
current and voltage behavior observed in the nodes

SPICE Netlist T
Cunit 4 0 50pF
Cmain y 0 90pF
M20 4 3 VDD VDD PMOS W=0.5u L=0.1u
M21 4 3 y 0 NMOS W=0.5u L=0.1u
R1 37 1 1k
R2 38 2 1k
M23 6 y VDD VDD PMOS W=0.1u L=0.1u
. . .
Voltage and Current Simulation Logs
//. . .
Desired Output Specification
The output voltage at node 'out' should be less than
360.0mV for Vin between 0V and 2V.
```

(b)

GPT-3.5-turbo (Assistant)

```
LLM Response:

**Identify the Trojan**
Based on the provided netlist and
simulation log, there is a Trojan circuit
present in the netlist. The Trojan is
inserted at nodes 3 and 4.
The Trojan is:
Cunit 4 0 50pF
Cmain y 0 90pF
M20 4 3 VDD VDD PMOS W=0.5u L=0.1u
M21 4 3 y 0 NMOS W=0.5u L=0.1u
M23 6 y VDD VDD PMOS W=0.1u L=0.1u
M24 6 y 0 0 NMOS W=0.1u L=0.1u
**Identify the Impacted Nodes**
The impacted nodes are 3, 4, 6, and y.
**Diagnosis Report**
In the Voltage Simulation Log, we can see
that for input voltages Vin between 0V and
2V, the output voltage at node 'out'
exceeds the desired specification of
360.0mV. Moreover, in the Current
Simulation Log X, we see that the current
through the impacted nodes (y and out)
shows abnormal behavior corresponding to
the input voltages where the output
voltage exceeds the desired specification.
. . .
```

(c)

Fig. 12. (a) Example prompt instructing the LLM with supervised learning rules for identifying Trojan-impacted nodes; few-shot examples are given to aid the LLM in the process of Trojan detection and localization. (b) Test prompt including the SPICE netlist, simulation log, and desired specifications provided to the LLM for analysis. (c) Response generated by LLM, highlighting the Trojan circuit in the netlist and Trojan-impacted nodes.

```
1.**Identify the Impacted Nodes**:  Suspect nodes: y, 4, 3,  2.
2. **Provide confidence score**: A confidence score is based on
the abnormal observations across these nodes.
**Node 'y'**: 1 (very high deviation, indicates short-circuit
behavior)
**Node '4'**: 0.70 (high deviation but less than 'y')
**Node '3'**: 0.85 (weak evidence but still indicates something)
**Node '2'**: 0.75 (interconnected with M25, affected by input
conditions)
Final confidence score = (1 + 0.70 + 0.85 + 0.75) / 4 =**0.825**
3. **List all input voltages that cause anomalies in impacted
nodes**:
\\ LLM response \\
4. **Comprehensive evaluation**: Voltage alteration above
thresholds in nodes 4 and 2 indicates abnormal thermal loading
or potentially unauthorized modifications affecting current
sources and leading to excessive heating or even latency.
```

Fig. 13. LLM-generated confidence scores.

left unmitigated, have a negligible effect on the circuit output behavior.

E. Comparison to ML-Based Trojan Detection Technique

As per our knowledge, efficient Trojan detection in analog circuits using circuit watermarking and neural twins (DAWN) [9] is the first work that uses ML in identifying Trojan hotspots. Its scope and methodology are significantly different from SPICED [1] and SPICED+, making direct comparison challenging. DAWN focuses on Trojan detection, while SPICED and SPICED+ further broaden the scope with Trojan localization and mitigation, respectively. Table X shows a comparison between the three methodologies in terms of their runtime overheads (preprocessing overhead + detection time). SPICED+, which uses end-to-end LLM inferencing for detection and diagnosis, achieves a significantly faster runtime compared with DAWN. It eliminates the preprocessing steps

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

PERFORMANCE OF LLMS IN DETECTING ANALOG TROJANS AND LOCATING TROJAN-IMPACTED NODES FOR SEVERAL ANALOG DESIGNS

Case	Netlist	Trojan identified?				Trojan Coverage (%)				Precision (%)				Recall (%)			
		FS	R	R_{FS}	R^*_{FS}	FS	R	R_{FS}	R^*_{FS}	FS	R	R_{FS}	R^*_{FS}	FS	R	R_{FS}	R^*_{FS}
1	642_troj_1	✓	✓	✓	✓	57.14	14.28	**100**	**100**	67	67	**100**	**100**	67	67	**100**	**100**
2	642_troj_6	✓	✗	✓	✓	28.57	0	**100**	**100**	43	0	**100**	**100**	100	0	**100**	**100**
3	642_troj_7	✗	✓	✓	✓	0	14.28	57.14	57.14	0	25	67	67	0	33.3	67	100
4	642_troj_20	✓	✓	✓	✓	71.5	57.14	100	85.7	43	12.5	100	100	67	33.3	100	67
5	642_troj_34	✓	✓	✓	✓	42.8	42.8	100	100	100	12.5	100	67	100	33.3	100	100
6	642_troj_36	✓	✗	✓	✓	57.14	0	57.14	42.8	43	67	100	67	33.3	67	100	100
7	642_troj_38	✓	✓	✓	✓	57.14	14.28	100	100	50	67	67	75	67	67	67	100
8	642_troj_39	✗	✓	✓	✓	0	28.57	85.7	85.7	0	33.3	100	100	0	33.3	100	100
9	755_troj_1	✓	✗	✓	✓	14.28	0	100	100	50	0	100	67	33.3	0	100	100
10	755_troj_2	✓	✗	✓	✓	14.28	0	85.7	85.7	43	0	75	75	67	0	100	100
11	755_troj_3	✓	✓	✓	✓	42.8	42.8	85.7	71.4	43	12.5	100	67	67	33.3	67	67
12	755_troj_5	✗	✓	✓	✓	0	42.8	71.4	100	25	12.5	100	100	67	33.3	100	67
13	755_troj_6	✗	✓	✓	✓	0	42.8	57.14	42.8	0	40	100	75	0	67	100	100
14	755_troj_9	✓	✓	✓	✓	57.14	57.14	57.14	57.14	50	33.3	100	100	67	33.3	100	67
15	755_troj_10	✓	✓	✓	✓	71.4	42.8	100	100	100	25	100	75	100	33.3	100	100
16	755_troj_11	✓	✓	✓	✓	57.14	42.8	85.7	100	71.4	0	100	100	33.3	0	100	100
17	755_troj_12	✓	✓	✓	✓	28.57	14.28	85.7	100	50	33.3	75	75	67	33.3	67	100
18	755_troj_16	✓	✓	✓	✓	57.14	14.28	71.4	85.7	67	0	75	100	67	0	100	100
19	755_troj_19	✓	✗	✓	✓	28.57	0	**100**	**100**	25	0	**100**	**100**	33.3	0	**100**	**100**
20	755_troj_23	✓	✓	✓	✓	28.57	28.57	71.4	71.4	50	33.3	75	100	67	33.3	100	100
21	755_troj_24	✗	✗	✓	✓	0	0	100	100	0	0	75	75	0	0	100	100
22	738_troj_3	✓	✓	✓	✓	14.28	14.28	85.7	71.4	67	40	100	100	67	67	67	100
23	738_troj_4	✓	✓	✓	✓	42.8	28.57	71.4	85.7	67	33.3	75	100	67	33.3	67	67
24	738_troj_7	✓	✗	✓	✓	57.14	0	100	100	25	0	100	67	33.3	0	67	67
25	738_troj_12	✓	✗	✓	✓	57.14	0	71.4	100	67	20	100	100	67	33.3	100	100
26	738_troj_13	✓	✗	✓	✓	57.14	0	85.7	71.4	40	0	75	75	67	0	100	100
27	738_troj_16	✓	✓	✓	✓	28.57	28.57	85.7	71.4	40	40	100	67	67	33.3	67	100
28	738_troj_17	✓	✗	✓	✓	71.4	0	57.14	71.4	0	0	75	100	0	0	100	100
29	738_troj_23	✓	✓	✓	✓	28.57	28.57	**100**	**100**	25	33.3	**100**	**100**	33.3	33.3	**100**	**100**
30	738_troj_26	✓	✓	✓	✓	14.28	28.57	100	85.7	40	33.3	75	100	67	33.3	100	100
Average						36.18	20.93	**84**	**80.6**	43.04	22.47	**90.3**	**86.46**	52.39	27.81	**92.3**	**93.4**

The proposed framework detects Trojan-impacted nodes as well as the Trojan circuit with 100% accuracy and zero false positives for the **highlighted** cases. Across all evaluated scenarios, applying supervised-learning rules combined with few-shot learning yields higher average Trojan coverage, precision, and recall (marked in **bold**).

TABLE VIII

EVALUATION OF SPICED ON GPT-4O-MINI MODEL

Case	Netlist	Trojan Coverage (%)	Recall (%)	FPR (%)
1	642_troj_6	85.7	67	0
2	642_troj_7	71.4	67	0
3	642_troj_20	57.1	100	0
4	642_troj_39	57.1	67	7.14
5	755_troj_5	85.7	67	7.14
6	755_troj_11	85.7	100	0
7	755_troj_12	100	100	7.14
8	755_troj_16	71.4	100	0
9	755_troj_23	100	100	7.14
10	738_troj_3	85.7	100	7.6
11	738_troj_4	85.7	100	7.6
12	738_troj_7	71.4	100	15.3
13	738_troj_23	57.1	100	7.6
14	738_troj_26	71.4	100	0
15	bg_troj_11	85.71	100	10.63
16	bg_troj_77	57.14	67	4.25
17	bg_troj_76	57.14	100	4.25
18	bg_troj_13	71.4	100	0
19	bg_troj_39	85.71	67	6.3
20	bg_troj_105	85.71	100	6.3
21	bg_troj_68	71.4	100	2.1
22	ldo_troj_76	71.4	67	7.6
23	ldo_troj_19	85.71	100	7.6
24	ldo_troj_117	85.71	100	0
25	ldo_troj_17	57.14	67	9.6
26	ldo_troj_13	85.71	100	7.6

used in DAWN, such as model training or converting circuits into channel-connected blocks (CCBs).

F. Resource Overhead

For syntax detection on the AMSNet dataset, the average prompt size was 796 tokens. For bandgap filter, the token size was 6000 tokens due to larger netlist size. For Trojan detection on the AMSNet dataset, the average token count is 13 500, while for bandgap filter and LDO, the token count is 95 500. The computational overhead for SPICED+ depends on the number of input tokens (supervised learning rules + few-shot examples + test netlist + task definitions) and output tokens. For GPT-3.5-turbo, input costs are approximately \$3 per 1M tokens, while output costs are around \$4 per 1M tokens [33]. In contrast, GPT-4o-mini is much more cost-effective, with input costs around \$0.15 for 1M tokens and output costs at \$0.6 per 1M tokens [33]. GPT-3.5-turbo supports up to 16 385 tokens, whereas GPT-4o-mini handles up to 128 K.

SPICED+ shows an average inferencing time of 20.2 s using GPT-4o-mini for the evaluated circuits. For larger netlists like bandgap filter and LDO, the prompt size increased the response times, but had minimal impact on Trojan detection accuracy. Note that there will be significant reduction in inference time if GPUs are used. Currently, LLMs are deployed on graphics processing unit (GPU) servers to facilitate inferencing, and our framework is platform-agnostic in this regard. This enables SPICED+ to adapt to various hardware accelerators such as GPUs and tensor processing unit (TPUs), depending on user deployment preferences.

VII. DISCUSSION AND FUTURE WORK

A. How Can We Leverage LLMs to Handle More Complex Circuits Such as an SAR-ADC?

For larger netlists such as successive-approximation-register analog-to-digital converter (SAR-ADC), it is not feasible to feed the entire netlist to the LLM as it might degrade the model performance, leading to inaccurate identification of the Trojan-impacted nodes. In such cases, a more efficient strategy would be to split the flattened SPICE netlist into CCBs using Synopsys Custom Compiler tool [34]. A CCB groups transistors and passive elements that share electrical connectivity into a single block. Each CCB can be represented as a graph, where the nodes represent components and the edges represent the connections between them. We provide the CCB connectivity graph and individual CCB graphs as inputs to the LLM. However, implementing this approach would require substantial modifications to our current framework and falls outside the scope of this work.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
COMPREHENSIVE EVALUATION OF SPICED AND SPICED+ FRAMEWORKS

Case	Circuit	Trojan Coverage (%)			FPR_{tr} (%)			Computation time (s)			n_{spec}	TMR (%)
		GPT-4o	[1]	**SPICED+**	GPT-4o	[1]	**SPICED+**	GPT-4o	[1]	**SPICED+**		
1	642_troj_6	14.2	85.7	100	17.2	10.3	6.8	8.8	9.1	21.3	10	100
2	642_troj_7	28.5	71.4	100	13.7	10.3	0	8.8	9.1	21.2	8	100
3	642_troj_20	28.5	57.1	100	13.7	0	0	8.7	9.2	19.1	11	100
4	642_troj_34	14.2	57.1	100	17.2	0	0	8.8	10.3	27.3	8	100
5	642_troj_36	28.5	71.4	85.7	17.2	6.8	4.3	8.8	10.2	23.2	8	100
6	642_troj_39	14.2	57.1	100	17.2	6.8	0	9.3	10.2	27.2	7	100
7	755_troj_1	14.2	57.1	100	13	13	4.3	9.1	10.8	17.3	10	100
8	755_troj_2	28.5	57.1	85.7	17.3	13	4.3	10.3	10.1	15.4	8	100
9	755_troj_5	28.5	85.7	85.7	13	8.6	0	9.4	9.1	18.7	9	100
10	755_troj_16	14.2	71.4	85.7	13	8.6	0	8.2	9.8	18.2	7	83.3
11	755_troj_19	14.2	85.7	100	13	0	0	8.6	10.8	16.9	9	100
12	755_troj_23	28.5	100	100	17.3	0	0	9.7	9.8	19.2	8	100
13	738_troj_3	14.2	85.7	85.7	16.6	4.3	0	8.5	9.5	19.7	9	100
14	738_troj_4	28.5	85.7	85.7	16.6	4.3	0	8.4	8.6	18.5	7	83.3
15	738_troj_7	28.5	71.4	71.4	20.8	0	0	8.4	12.8	20.6	8	100
16	738_troj_12	28.5	71.4	85.7	16.6	4.1	4.1	9.3	11.6	19.5	9	83.3
17	738_troj_13	28.5	71.4	100	16.6	4.1	0	8.8	9.1	21.8	8	83.3
18	738_troj_23	14.2	57.1	85.7	20.8	4.1	4.1	9.2	9.2	22.7	8	100
19	bg_troj_11	14.2	85.7	100	10.6	3.9	3.9	9.5	12.3	23.6	9	100
20	bg_troj_77	28.5	57.1	100	8.5	3.9	0	10.8	12.2	22.5	10	100
21	bg_troj_13	28.5	71.4	85.7	10.6	5.8	0	10.1	13.4	23.6	9	83.3
22	bg_troj_39	14.2	85.7	100	14.3	3.9	4.6	9.4	12.4	20.4	9	83.3
23	bg_troj_105	14.2	85.7	100	14.3	3.9	0	10.2	13.5	22.7	10	100
24	bg_troj_68	14.2	71.4	100	10.6	3.9	0	9.4	12.1	25.4	11	100
25	bg_troj_76	28.5	57.1	71.4	10.6	3.9	3.9	10.2	10.2	20.7	9	100
26	ldo_troj_76	14.2	71.4	85.7	2.1	1.2	1.2	10.2	12.7	21.9	9	100
27	ldo_troj_19	14.2	85.7	100	3.6	2	0	9.2	13.2	22.1	8	83.3
28	ldo_troj_117	28.5	85.7	85.7	2.1	1.2	0	9.1	14.2	22.6	10	100
29	ldo_troj_17	28.5	57.1	71.4	3.6	1.2	2	10.5	11.3	25.7	10	75
30	ldo_troj_13	14.2	85.7	100	3.6	2.5	2	10.5	10.2	25.7	10	83.3
Average		22.3	73.3	**93.3**	13	4.3	**1.4**	9.2	11.1	20.2	≈ 9	**94.6**

TABLE X
COMPARISON OF RUNTIME OVERHEADS OF SPICED AND SPICED+ WITH DAWN [9] (t_d: DETECTION TIME)

Method	Trojan Detection?	Trojan Diagnosis?	Preprocessing		t_d (s)
			Type	Time (s)	
DAWN [9]	✓	✗	Split into CCB Parse log files Neural twin training	30 10 55	0.028
SPICED	✓	✓	Format log files	7.8	12.2
SPICED+	✓	✓	Format log files	7.8	22.5

B. Can AC Analysis Be Incorporated in SPICED+?

While not evaluated extensively in this work, parasitic effects can impact the node behavior, specifically in high-frequency analog designs. Even when the output voltage remains within the desired specification, the parasitic effect can impact the ac performance. By incorporating ac analysis in the SPICED workflow, we can extensively evaluate the frequency-domain behavior of suspect nodes and obtain the correlation between the parasitic components and the node voltages under different input voltages.

VIII. CONCLUSION

We have shown the effectiveness of LLMs for Trojan mitigation in analog designs. By applying supervised learning rules for identifying Trojan-impacted nodes and iteratively correcting the netlist, we achieve an average TMR of 94.6%, an average FPR_{tr} of 1.4%, and an average Trojan coverage of 93.3% for the evaluated benchmark circuits.

REFERENCES

[1] J. Chaudhuri, D. Thapar, A. Chaudhuri, F. Firouzi, and K. Chakrabarty, "SPICED: Syntactical bug and trojan pattern identification in a/ms circuits using llm-enhanced detection," in *Proc. IEEE Phys. Assurance Inspection Electron. (PAINE)*, Nov. 2024, pp. 1–7.

[2] M. M. Alam et al., "Challenges and opportunities in analog and mixed signal (AMS) integrated circuit (IC) security," *J. Hardw. Syst. Secur.*, vol. 2, no. 1, pp. 15–32, Mar. 2018.

[3] M. Rostami, F. Koushanfar, and R. Karri, "A primer on hardware security: Models, methods, and metrics," *Proc. IEEE*, vol. 102, no. 8, pp. 1283–1295, Aug. 2014.

[4] B. Bilgic and S. Ozev, "Performance degradation monitoring for analog circuits using lightweight built-in components," in *Proc. IEEE 40th VLSI Test Symp. (VTS)*, Apr. 2022, pp. 1–7.

[5] K. Yang, M. Hicks, Q. Dong, T. Austin, and D. Sylvester, "A2: Analog malicious hardware," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2016, pp. 18–37.

[6] N. Gupta, M. S. Desai, M. Wijtvliet, S. Rai, and A. Kumar, "DELTA: DEsigning a stealthy trigger mechanism for analog hardware trojans and its detection analysis," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 787–792.

[7] M. Abedi, T. Yang, Y. Fei, and A. Shrivastava, "High-precision nano-amp current sensor and obfuscation based analog trojan detection circuit," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2022, pp. 3324–3328.

[8] J. Talukdar, A. Chaudhuri, M. Bhattacharya, and K. Chakrabarty, "Automatic structural test generation for analog circuits using neural twins," in *Proc. ITC*, Sep. 2022, pp. 145–154.

[9] J. Chaudhuri, M. Bhattacharya, and K. Chakrabarty, "DAWN: Efficient trojan detection in analog circuits using circuit watermarking and neural twins," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 10, pp. 2930–2943, Oct. 2024.

[10] S. Thakur et al., "VeriGen: A large language model for verilog code generation," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 3, pp. 1–31, May 2024.

[11] R. Zhong et al., "LLM4EDA: Emerging progress in large language models for electronic design automation," 2023, *arXiv:2401.12224*.

[12] Y. Yin, Y. Wang, B. Xu, and P. Li, "ADO-LLM: Analog design Bayesian optimization with in-context learning of large language models," 2024, *arXiv:2406.18770*.

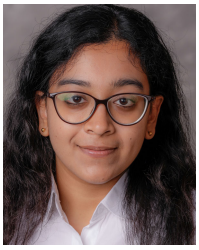
[13] Y. Lai et al., "AnalogCoder: Analog circuit design via training-free code generation," 2024, *arXiv:2405.14918*.

[14] A. Hammoud et al., "Human language to analog layout using Glayout layout automation framework," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–7.

[15] Z. Tao et al., "AMSNet: Netlist dataset for AMS circuits," 2024, *arXiv:2405.09045*.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.

[16] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE 5th Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2023, pp. 1–6.
[17] M. Liu et al., "ChipNeMo: Domain-adapted LLMs for chip design," 2023, *arXiv:2311.00176*.
[18] S. Thakur et al., "Benchmarking large language models for automated verilog RTL code generation," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Apr. 2023, pp. 1–6.
[19] J. Bhandari, R. Sadhukhan, P. Krishnamurthy, F. Khorrami, and R. Karri, "SENTAUR: Security EnhaNced trojan assessment using LLMs against undesirable revisions," 2024, *arXiv:2407.12352*.
[20] Y.-D. Tsai, M. Liu, and H. Ren, "RTLFixer: Automatically fixing RTL syntax errors with large language models," 2023, *arXiv:2311.16543*.
[21] H. Wu et al., "Chateda: A large language model powered autonomous agent for eda," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 10, pp. 3184–3197, Oct. 2024.
[22] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4043–4057, 2024.
[23] G. Kokolakis, A. Moschos, and A. D. Keromytis, "Harnessing the power of general-purpose LLMs in hardware trojan design," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, Cham, Switzerland, 2024, pp. 176–194.
[24] D. Saha, K. Yahyaei, S. Kumar Saha, M. Tehranipoor, and F. Farahmandi, "Empowering hardware security with LLM: The development of a vulnerable hardware database," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, May 2024, pp. 233–243.
[25] B. Rozière et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
[26] D. A. Tomassi, "Bugs in the wild: Examining the effectiveness of static analyzers at finding real-world bugs," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Oct. 2018, pp. 980–982.
[27] N. Ayewah, W. Pugh, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE Softw.*, vol. 25, no. 5, pp. 22–29, Sep. 2008.
[28] H. S. Choo et al., "Machine-Learning-Based multiple abstraction-level detection of hardware trojan inserted at register-transfer level," in *Proc. IEEE 28th Asian Test Symp. (ATS)*, Dec. 2019, pp. 98–980.
[29] R. Yasaei, S.-Y. Yu, and M. A. Al Faruque, "GNN4TJ: Graph neural networks for hardware trojan detection at register transfer level," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1504–1509.
[30] Infineon. *Adsbenchmark*. Accessed: Jul. 20, 2024. [Online]. Available: https://rb.gy/1m2263
[31] A. Vladimirescu, *The Spice Book*. Hoboken, NJ, USA: Wiley, 1994.
[32] M. Tehranipoor et al., "Trustworthy hardware: Trojan detection and design-for-trust challenges," *Computer*, vol. 44, no. 7, pp. 66–74, Jul. 2011.
[33] OpenAI. *Pricing*. Accessed: Jul. 20, 2024. [Online]. Available: https://t.ly/ibOnf
[34] Synop. *Custom Compiler*. Accessed: Jul. 20, 2024. [Online]. Available: http://bit.ly/40MNuXS



Jayeeta Chaudhuri (Member, IEEE) received the B.E. degree from the Department of Electronics and Telecommunication Engineering, Jadavpur University, Kolkata, India, in 2020, and the M.S. degree from Duke University, Durham, NC, USA, in 2023. She is currently working toward the Ph.D. degree at Arizona State University, Tempe, AZ, USA, and Duke University.

During her Ph.D. degree, she has done research internships at Synopsys, Sunnyvale, CA, USA, and Intel, Hillsboro, OR, USA. Her current research interests include cloud FPGA security and analog IC security.



Dhruv Thapar received the B.Tech. and M.Tech. degrees from IIT Kharagpur, Kharagpur, India, in 2020. He is currently working toward the Ph.D. degree in electrical and computer engineering at Arizona State University, Tempe, AZ, USA.

He was an Intern with Intel Corporation, Hillsboro, OR, USA. His current research interests include design-for-testability, fault tolerance, and resilience of emerging technologies, such as ferroelectric-FETs, AI hardware, and 2.5-D/3-D systems based on advanced packaging.



Arjun Chaudhuri received the B.Tech. degree in electronics and electrical communication engineering from IIT Kharagpur, Kharagpur, India, in 2017, and the M.S. and Ph.D. degrees in electrical and computer engineering from Duke University, Durham, NC, USA, in 2020 and 2022, respectively.

He is currently a Senior Design-for-X (DFX) Methodology Engineer with NVIDIA Corporation, Santa Clara, CA, USA, and an Adjunct Faculty with Arizona State University, Tempe, AZ, USA. His research interests span design-for-testability and safety-criticality analysis for AI accelerators, test and diagnosis of 3-D SoCs, and applications of AI in test and security of AMS circuits.



Farshad Firouzi is currently an Adjunct Faculty Member with Arizona State University, Tempe, AZ, USA, and Duke University, Durham, NC, USA. He has authored more than 60 conference papers and journal articles. He is the Editor/Author of the book "*Intelligent Internet of Things: From Device to Fog and Cloud*."

Dr. Firouzi has also served as an Associate Editor and a Guest Editor for IEEE INTERNET OF THINGS JOURNAL, IEEE TRANSACTIONS ON VLSI SYSTEMS, IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, *Future Generation Computer Systems*, *Journal of Network and Computer Applications*, and *Information Systems*.



Krishnendu Chakrabarty (Fellow, IEEE) received the B.Tech. degree from Indian Institute of Technology Kharagpur, Kharagpur, India, in 1990, and the M.S.E. and Ph.D. degrees from the University of Michigan at Ann Arbor, Ann Arbor, MI, USA, in 1992 and 1995, respectively.

He is currently the Fulton Professor of Microelectronics with the School of Electrical, Computer and Energy Engineering, Arizona State University (ASU), Tempe, AZ, USA, where he is also the Chief Technology Officer (CTO) with the Department of Defense Microelectronics Commons Southwest Advanced Prototyping (SWAP) Hub and the Director of the ASU Center on Semiconductor Microelectronics.

Prof. Chakrabarty is a fellow of ACM and American Association for the Advancement of Science (AAAS), and a Golden Core Member of the IEEE Computer Society.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on June 08,2026 at 08:33:44 UTC from IEEE Xplore. Restrictions apply.